

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Facultad de Ingeniería
Ingeniería en Ciencias y Sistemas

MANUAL TÉCNICO

Práctica 2 - Quetzal Space Defender

Proyecto	Quetzal Space Defender - Side Scroller
Nombre	Rene Alejandro Fuentes Ramos
Carné	202501994
Curso	Introducción a la Programación y Computación 1
Sección	C
Semestre	Segundo semestre 2026

Fecha de documentación: 23/09/2026

Contenido

1. Descripción general
2. Herramientas y estructura
3. Clases principales
4. Funcionamiento del juego
5. Resultados y reporte
6. Métodos importantes
7. Pruebas recomendadas
8. Observación para la entrega

1. Descripción general

Quetzal Space Defender es un juego de desplazamiento horizontal hecho en Java. El jugador registra un piloto, elige una nave, esquiva enemigos, dispara y recoge objetos especiales. Al perder sus tres vidas se registra el resultado de la partida para consultarlo en el top, el historial y un reporte HTML.

El proyecto usa programación orientada a objetos, arreglos, ciclos, hilos y una interfaz construida con Swing. El diseño visual es sencillo porque el objetivo principal es demostrar el funcionamiento de las mecánicas.

2. Herramientas y estructura

Recurso	Uso
Java	Lógica del juego y clases
Swing y AWT	Ventanas, botones, teclado y dibujo
Thread	Movimiento independiente de proyectiles y enemigos
<code>javax.swing.Timer</code>	Actualización del panel, generación de objetos y efectos temporales
JFreeChart	Gráfica de mejores puntajes
<code>java.io</code>	Escritura del reporte HTML
ImageIO	Exportación de la gráfica a PNG
Maven	Dependencia de JFreeChart y compilación

La aplicación está dentro de la carpeta Practica2. El código Java pertenece al paquete `quetzal`, ubicado normalmente en `src/main/java/quetzal/`. El archivo `pom.xml` contiene la configuración de Maven y la dependencia `org.jfree:jfreechart:1.5.6`.

Para abrir el proyecto en otro equipo se necesita un JDK compatible con la versión indicada en `maven.compiler.release` del `pom.xml`. En la configuración utilizada durante el desarrollo se estableció Java 26.

3. Clases principales

Clase	Responsabilidad
Piloto	Guarda nombre y tipo de nave
GestionPilotos	Registra, valida y consulta pilotos mediante un arreglo
MenuPrincipal	Muestra las opciones principales
VentanaCrearPiloto	Recibe los datos de un piloto nuevo
VentanaSeleccionPiloto	Permite elegir un piloto antes de jugar
VentanaJuego	Contiene el panel del juego y los datos visibles del piloto
PanelJuego	Coordina controles, dibujo, colisiones, puntaje, vidas y final de partida
Proyectil	Representa un disparo que avanza en su propio hilo
Enemigo	Representa una nave enemiga que avanza en su propio hilo
ObjetoEspacial	Representa Snitch, Bludger o Quaffle
Partida	Guarda piloto, nave, puntaje y fecha de un resultado
GestionPartidas	Almacena y ordena las partidas en un arreglo
VentanaTopPuntajes	Muestra el top, la gráfica y el historial
GeneradorReporte	Escribe el reporte HTML y exporta su gráfica

La clase de arranque crea una sola instancia de `GestionPilotos` y una de `GestionPartidas`. Las ventanas reciben esas mismas instancias para compartir la información mientras el programa está abierto.

4. Funcionamiento del juego

Pilotos y dificultad

`GestionPilotos` guarda los pilotos en un arreglo y lleva un contador de posiciones ocupadas. Al registrar se comprueba que el nombre no esté vacío, que no se repita y que exista espacio disponible.

Nave	Movimiento	Espera entre disparos
Explorador	Rápido	2 segundos
Caza Estelar	Medio	1 segundo
Acorazado	Lento	0.3 segundos

Movimiento y disparos

PanelJuego registra si cada tecla de dirección está presionada. Un Timer de Swing revisa esos estados aproximadamente cada 16 ms y actualiza la posición de la nave. Como los ejes horizontal y vertical se comprueban por separado, la nave también puede moverse en diagonal. La posición se limita al área visible del panel.

Al pulsar espacio, el panel comprueba la recarga correspondiente a la nave y crea un **Proyectil**. Cada proyectil es un hilo independiente: modifica su coordenada X hasta salir de la pantalla o ser detenido por una colisión. Los proyectiles inactivos se retiran del arreglo.

Enemigos y colisiones

Los enemigos se crean periódicamente en el borde derecho y aparecen a distintas alturas. Cada Enemigo tiene su propio hilo para desplazarse hacia la izquierda.

El panel revisa si se superponen las áreas rectangulares de un proyectil y un enemigo. Si chocan, los dos se desactivan. Si un enemigo toca la nave, desaparece, se pierde una vida y se aplica un breve período de protección para no perder varias vidas por el mismo choque. Al llegar a cero vidas, el juego termina y se registra la partida.

Destruir enemigos no suma puntos en esta implementación; los puntos se obtienen de los objetos especiales.

Objetos especiales

Objeto	Efecto al tocar la nave
Quaffle (azul)	Suma 10 puntos
Snitch (amarillo)	Suma 150 puntos y elimina los enemigos visibles
Bludger (gris)	Bloquea el movimiento durante 2 segundos

Los objetos aparecen desde la derecha. La aparición del Snitch es menos frecuente que la de los otros tipos.

5. Resultados y reporte

Al terminar una partida, PanelJuego crea un objeto Partida y lo envía a GestionPartidas. Esta gestión conserva el orden original en su arreglo. Para el top, crea una copia y la ordena de mayor a menor puntaje mediante el método burbuja; así el historial no pierde su orden de registro.

VentanaTopPuntajes presenta dos pestañas: el top con todas las partidas ordenadas y una gráfica de hasta diez resultados, y el historial con todas las partidas en el orden en que se jugaron.

GeneradorReporte crea la carpeta reportes/ al solicitar el reporte, escribe reporte_partidas.html usando FileWriter y PrintWriter, y guarda grafica_puntajes.png con ImageIO. El HTML presenta el historial y la imagen de la gráfica; desde el navegador puede imprimirse como PDF. La imagen y el HTML deben permanecer juntos para que la gráfica se vea correctamente.

Almacenamiento: los arreglos conservan pilotos y partidas durante la ejecución. Esta versión no los recupera tras cerrar el programa. El HTML exportado es una copia de consulta, no una base de datos que se cargue al iniciar.

6. Métodos importantes

Método o bloque	Función
Registro de GestionPilotos	Valida y agrega un piloto al arreglo
cambiarEstadoTecla	Actualiza qué direcciones o teclas están presionadas
actualizarPosicion y validarLimites	Mueven la nave y evitan que salga del panel
disparar	Revisa la recarga y arranca el hilo del proyectil
generarEnemigo	Crea un enemigo en el borde derecho
Detección de colisiones	Desactiva objetos que chocan y aplica efectos
finalizarPartida	Detiene la partida y registra el resultado una sola vez
obtenerPartidasOrdenadas	Devuelve una copia ordenada para el top
generarReporte	Genera el HTML y la imagen de la gráfica

7. Pruebas recomendadas

- Registrar pilotos con nombres válidos, vacíos y repetidos
- Probar las tres naves, sus velocidades y sus tiempos de recarga
- Moverse en diagonal y verificar que la nave no salga de la ventana
- Disparar a un enemigo y comprobar que ambos desaparezcan
- Recoger cada objeto especial y verificar su efecto
- Perder tres vidas, verificar el mensaje y volver al menú
- Jugar varias partidas y comparar top, historial y gráfica
- Generar el HTML, abrirlo y comprobar que cargue su imagen

8. Observación para la entrega

Este manual describe la versión desarrollada durante la práctica. Antes de la entrega conviene ejecutar **Clean and Build** en NetBeans y revisar que el nombre de la clase principal configurada en `pom.xml` coincida con la clase que realmente contiene `main`.

Evidencia visual de la interfaz

Las siguientes pantallas muestran el juego y el módulo de resultados en funcionamiento

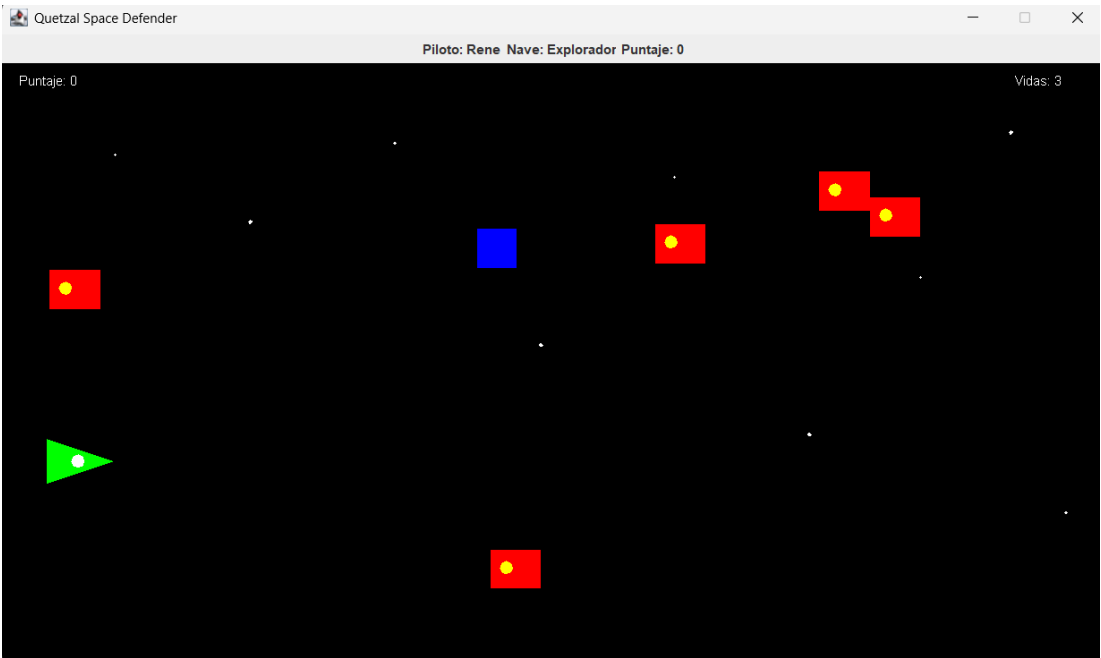


Figura 1. Partida en ejecución, HUD y enemigos.

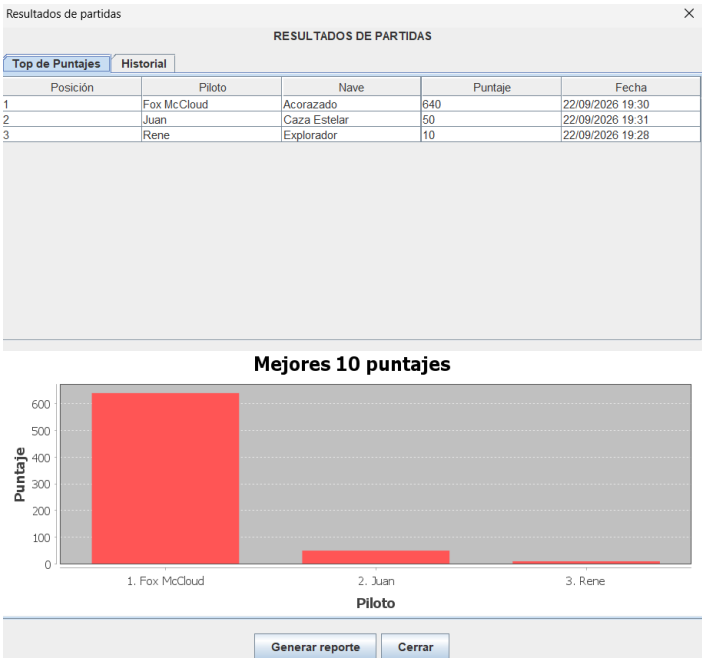


Figura 2. Tabla y gráfica generada con JFreeChart.